

```

manchodu@srini:/tmp
[1]+  Done                  sleep 30
manchodu@srini:/tmp$ cat /proc/12345/status | grep State
cat: /proc/12345/status: No such file or directory
manchodu@srini:/tmp$ cd /tmp
manchodu@srini:/tmp$ printf '#include <stdio.h>\n#include <stdlib.h>\n#include <unistd.h>\n#include <sys/wait.h>\n\nint main() {\n    int n = 3;\n    pid_t pids[3];\n\n    for (int i = 0; i < n; i++) {\n        pid_t pid = fork();\n        if (pid == 0) {\n            printf("Child %d: PID=%d\n", i, getpid());\n            sleep(i + 1);\n            exit(i);\n        } else {\n            pids[i] = pid;\n        }\n    }\n\n    for (int i = 0; i < n; i++) {\n        int status; waitpid(pids[i], &status, 0);\n        printf("Reaped child PID=%d, exit code=%d\n", pids[i], WEXITSTATUS(status));\n    }\n    return 0;\n}' > task4a.c
manchodu@srini:/tmp$ cat task4a.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int n = 3;
    pid_t pids[3];

    for (int i = 0; i < n; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            printf("Child %d: PID=%d\n", i, getpid());
            sleep(i + 1);
            exit(i);
        } else {
            pids[i] = pid;
        }
    }

    for (int i = 0; i < n; i++) {
        int status;
        waitpid(pids[i], &status, 0);
        printf("Reaped child PID=%d, exit code=%d\n", pids[i], WEXITSTATUS(status));
    }

    return 0;
}
manchodu@srini:/tmp$

```

```
manchodu@srini:/tmp
manchodu@srini:/tmp$ cat task4a.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int n = 3;
    pid_t pids[3];

    for (int i = 0; i < n; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            printf("Child %d: PID=%d\n", i, getpid());
            sleep(i + 1);
            exit(i);
        } else {
            pids[i] = pid;
        }
    }

    for (int i = 0; i < n; i++) {
        int status;
        waitpid(pids[i], &status, 0);
        printf("Reaped child PID=%d, exit code=%d\n", pids[i], WEXITSTATUS(status));
    }

    return 0;
}

manchodu@srini:/tmp$ gcc task4a.c -o task4a
manchodu@srini:/tmp$ ./task4a
Child 0: PID=1352
Child 1: PID=1353
Child 2: PID=1354
Reaped child PID=1352, exit code=0
Reaped child PID=1353, exit code=1
Reaped child PID=1354, exit code=2
manchodu@srini:/tmp$
```

```

manchodu @ srini: /tmp
}
manchodu @ srini:/tmp$ gcc task4a.c -o task4a
manchodu @ srini:/tmp$ ./task4a
Child 0: PID=1352
Child 1: PID=1353
Child 2: PID=1354
Reaped child PID=1352, exit code=0
Reaped child PID=1353, exit code=1
Reaped child PID=1354, exit code=2

manchodu @ srini:/tmp$ printf '#include <stdio.h>\n#include <unistd.h>\n\nmint main() {\n
    pid_t pid = fork();\n    if (pid == 0) {\n        exit(0);\n    } else {\n        p
printf("Parent sleeping, child PID=%d is now a zombie\n", pid);\n        sleep(20)
;\n    }\n    return 0;\n}\n' > task4b_zombie.c

manchodu @ srini:/tmp$ gcc task4b_zombie.c -o task4b_zombie
task4b_zombie.c: In function 'main':
task4b_zombie.c:7:9: warning: implicit declaration of function 'exit' [-Wimplicit-function-
declaration]
    7 |         exit(0);
      |         ^~~~~
task4b_zombie.c:3:1: note: include '<stdlib.h>' or provide a declaration of 'exit'
    2 | #include <unistd.h>
    +++ | #include <stdlib.h>
    3 |
task4b_zombie.c:7:9: warning: incompatible implicit declaration of built-in function 'exit'
[-Wbuiltin-declaration-mismatch]
    7 |         exit(0);
      |         ^~~~~
task4b_zombie.c:7:9: note: include '<stdlib.h>' or provide a declaration of 'exit'
manchodu @ srini:/tmp$ ./task4b_zombie &
[1] 1374
manchodu @ srini:/tmp$ Parent sleeping, child PID=1375 is now a zombie
manchodu @ srini:/tmp$ ps aux | grep Z
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
manchodu  1375  0.0  0.0      0     0 pts/0    Z   04:10   0:00 [task4b_zombie] <defunct>
manchodu  1377  0.0  0.0  4096  2104 pts/0    S+  04:10   0:00 grep --color=auto Z

manchodu @ srini:/tmp$ printf '#include <stdio.h>\n#include <unistd.h>\n#include <sys/wait.
h>\n\nmint main() {\n    pid_t pid = fork();\n    if (pid == 0) {\n        exit(0);\n
    } else {\n        wait(NULL);\n        printf("Child reaped, no zombie now\n");\n    }
    return 0;\n}\n' > task4c_wait.c

```